МИНИСТЕРСТВО ОБРАЗОВАНИЯ СВЕРДЛОВСКОЙ ОБЛАСТИ

ГАПОУ СО «ЕКАТЕРИНБУРГСКИЙ МОНТАЖНЫЙ КОЛЛЕДЖ»

КУРСОВОЙ ПРОЕКТ

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

Первый этап: REST-сервис на FastAPI

По дисциплине «МДК 05.03 Тестирование информационных систем»

Тема: «Создание и тестирование АРМ для обработки информации об участии студентов в олимпиадном движении, конкурсах профессионального мастерства»

Выполнил: Пеннер Андрей Владимирович

Группа: ИП-41 / ИП-41к (уточнить)

Номер студенческого: _____

Руководитель: _____

Подписи: _____

Оценка: _____

Екатеринбург
2026

ВЕДОМОСТЬ ПРОЕКТА

Таблица 1 — Состав комплекта первого этапа

№	Наименование	Представление
1	Пояснительная записка	docs/ Пояснительная_записка .docx и PDF
2	REST API FastAPI v1/v2	app/
3	Автоматические тесты	tests/ и pytest.ini
4	Зависимости и установка	requirements*.txt, database.sql, init_db.py, README
5	Руководства API и установки	docs/API.md, docs/INSTALL.md
6	Тестовая документация	docs/TEST_PLAN.md, TEST_CASES.md, TEST_REPORT.md

Комплект содержит исходный REST-сервис, тесты, зависимости и документацию первого этапа. Конфигурация .env, виртуальное окружение и рабочие пользовательские файлы в Git не включаются. Учебные записи и документы создаются командой python init_db.py только в пустой базе.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ · · · 4
1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ · · · 5
1.1 Уточнения и критерии приёмки · · · 6
2 АНАЛИТИЧЕСКАЯ ЧАСТЬ · · · 7
2.1 Цели, функции и задачи системы · · · 7
2.2 Описание предметной области · · · 8
2.3 Пользователи и разграничение доступа · · · 9
2.4 Внутренняя архитектура · · · 11
2.5 Проектные решения по тестированию · · · 12
3 ПРОЕКТНАЯ ЧАСТЬ · · · 13
3.1 Стек технологий · · · 13
3.2 Модель и ограничения базы данных · · · 14
users · · · 15
olympiads · · · 15
participations · · · 16
attachments · · · 16
Связи и ограничения · · · 17
3.3 Файловая структура и назначение модулей · · · 18
3.4 Алгоритм записи на олимпиаду · · · 19
3.4.1 Подготовка, отзыв и публикация протокола · · · 21
3.4.2 Аутентификация и отзыв токенов · · · 22
3.5 Документация REST API · · · 24
3.5.1 Примеры запросов и обработка ответа · · · 26
3.5.2 Работа пользователей с API · · · 27
3.5.3 Руководство по развёртыванию · · · 28
3.5.4 Хранение данных и резервное копирование · · · 29
3.6 Результаты тестирования · · · 30
4 ГЛОССАРИЙ · · · 32
5 СПИСОК ИСТОЧНИКОВ · · · 33
ЗАКЛЮЧЕНИЕ · · · 35
ПРИЛОЖЕНИЕ 1 — РЕЕСТР ДЕФЕКТОВ И ОГРАНИЧЕНИЙ · · · 36
ПРИЛОЖЕНИЕ 2 — ПРИМЕРЫ РАБОТЫ API · · · 37

ВВЕДЕНИЕ

Тема работы — создание и тестирование АРМ для обработки информации об участии студентов в олимпиадном движении и конкурсах профессионального мастерства. Для каждого мероприятия требуется хранить положение, даты, список участников, баллы и результаты. Раздельное ведение таблиц и документов приводит к повторному вводу и расхождениям при подготовке отчётов.

Актуальность проекта связана с необходимостью упорядочить эти сведения и разграничить ответственность пользователей. Студенту нужен доступ к своим заявкам и результатам, преподавателю — к подготовке документов своего мероприятия, администратору — к учётным записям и публикации проверенных протоколов. Гостю достаточно открытых сведений без доступа к редактированию и личным данным учётных записей.

Объект разработки — обработка сведений об участии студентов в олимпиадном движении и конкурсах профессионального мастерства. Предмет разработки — REST-сервис, который хранит эти сведения в PostgreSQL и предоставляет операции через HTTP. Основанием служит персональное техническое задание, выданное Пеннер Андрею Владимировичу [25].

Цель первого этапа — разработать и проверить самостоятельный REST-сервис на FastAPI. Для достижения цели требуется определить сущности и права пользователей; реализовать обработку заявок, документов и результатов; подготовить отчёты Word и Excel; проверить положительные и отрицательные сценарии; составить документацию по установке и использованию.

Работа ограничена REST API первого этапа со сроком до 15 декабря 2026 года. Django и его пользовательские страницы относятся ко второму этапу и здесь не разрабатываются. В записке рассматриваются задание, предметная область, устройство API, испытания и установка. Основание и оформление определены предоставленными документами [22, 23, 24, 25].

1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Полная тема проекта: «Создание и тестирование АРМ для обработки информации об участии студентов в олимпиадном движении, конкурсах профессионального мастерства». Разрабатываемый сервис предназначен для ввода, хранения и обработки сведений об олимпиадах, участниках и результатах. Он должен обеспечивать систематизацию информации и формирование отчётности.

Гость просматривает список олимпиад, отбирает их по году, получает карточку, положение, задания прошлых лет и опубликованные протоколы. Студент входит в систему, изменяет собственный профиль, подаёт заявки на несколько мероприятий и просматривает свои опубликованные результаты.

Администратор создаёт преподавателей, редактирует существующие учётные записи, заполняет карточки мероприятий, назначает ответственных и публикует подготовленные протоколы. Ему доступны протокол выбранной олимпиады и годовой отчёт. Преподаватель создаёт студентов, ведёт документы и результаты своей олимпиады, готовит протокол и при необходимости отзывает его до публикации.

В карточке студента хранятся ФИО, дата рождения, электронная почта, логин, учебное заведение, уровень образования, курс и специальность. Пароль принимается при создании, но в открытом виде не сохраняется. Для мероприятия задаются название, описание, ответственный преподаватель и даты. В заявке связываются студент и олимпиада, а после проверки работ сохраняются баллы и итоговая категория.

Обязательные технические условия первого этапа: FastAPI, SQLAlchemy, клиент-серверная PostgreSQL, JWT, версии v1/v2 и отдельная автоматическая документация. Для основных операций предусмотрены HTTP-методы GET, POST, PUT/PATCH и DELETE. Критерии приёмки включают выполнение сценария от заявки до публикации, создание настоящих DOCX/XLSX и проверку ограничений доступа [23].

1.1 Уточнения и критерии приёмки

В исходном перечне категорий отдельно не указан студент, хотя далее описаны действия зарегистрированного участника и создание студенческих учётных записей. Поэтому для участника принята роль student. Самостоятельная публичная регистрация не добавлена: студента создаёт преподаватель или администратор, преподавателя — администратор.

Задание не содержит общих порогов для назначения победителя и призёра. Категория результата вводится ответственным преподавателем по положению конкретной олимпиады. Система проверяет допустимое значение категории и корректность баллов, но не заменяет решение организатора произвольной формулой.

Приняты три состояния протокола: draft — формируется, prepared — подготовлен, published — опубликован. В prepared результаты и заявки заблокированы. Исправление возможно после отзыва в draft. Опубликованный протокол не отзывается, поскольку в ТЗ отзыв разрешён только для подготовленных протоколов.

Критерии приёмки: корректный цикл от заявки до публикации, разграничение ответственности, скрытие результатов до публикации, читаемые Word/Excel, не менее 50 записей на основную сущность и покрытие серверного кода не ниже 80%. Реестр требований и матрица находятся в docs/TEST_PLAN.md; действия и результаты — в docs/TEST_CASES.md и docs/TEST_REPORT.md.

2 АНАЛИТИЧЕСКАЯ ЧАСТЬ

2.1 Цели, функции и задачи системы

Главная задача системы — хранить согласованный набор сведений о мероприятиях и участии студентов. Данные вводятся один раз и используются при просмотре, публикации и выгрузке отчётов. Повторное ручное составление нескольких несвязанных таблиц заменяется запросами к общей базе.

Функции распределены между модулями авторизации, учётных записей, олимпиад, участия, протоколов и документов. Такое разделение соответствует действиям пользователей: учётная запись определяет исполнителя, карточка описывает мероприятие, заявка связывает его со студентом, протокол содержит итог, а вложения хранят положение и бланки.

Работа с API на первом этапе выполняется через Swagger или обычный HTTP-клиент. Отдельная клиентская программа не включена. Будущее веб-приложение сможет обращаться к тем же маршрутам, не изменяя данные прямыми запросами к таблицам.

Задачи обработки: отбор по году, поиск и пагинация v2, уникальность заявки, полнота протокола, права и выгрузки. Используются небольшие функции и несколько файлов: модели объединены в models.py, схемы — в schemas.py, обработчики — в main.py. Дополнительные уровни репозиториев и фоновые процессы для учебного объёма не вводятся.

2.2 Описание предметной области

Мероприятие имеет одного ответственного преподавателя и может иметь много участников. Студент может участвовать в нескольких мероприятиях. Такая связь «многие ко многим» раскрыта через таблицу participations. В ней находятся баллы и результат конкретного студента на конкретной олимпиаде.

Протокол не выделен в отдельную таблицу: у мероприятия существует один текущий набор результатов и один статус. Это уменьшает число сущностей без потери функций первого этапа. История версий протокола и повторное опубликование исправлений не входят в выданное ТЗ.

Документ относится к одной олимпиаде и имеет вид: положение, задание, сертификат или грамота. Количество вложений не ограничивается одним бланком. Сами файлы находятся в storage, метаданные — в attachments. При замене сохраняется ID документа, содержимое записывается под новым случайным именем, старый файл удаляется после успешного сохранения.

Исходные данные определены персональным ТЗ и методическими материалами [23, 24, 25]. Реальные результаты и опросы сотрудников не предоставлялись. В пустой базе инициализация создаёт 50 пользователей, 50 олимпиад, 50 участий, 50 документов. Пользователи: один администратор, два преподавателя и 47 студентов. Все сведения вымышлены. Реляционные связи отделяют личные данные от результатов участия [1, 3, 4, 9, 20].

2.3 Пользователи и разграничение доступа

Гость не имеет учётной записи и получает только общедоступные сведения. Студент работает со своим профилем и своими заявками. Преподаватель создаёт студенческие аккаунты и отвечает за мероприятия, где его идентификатор указан в teacher_id. Администратор обслуживает всю систему и выполняет публикацию.

Аутентификация подтверждает личность по логину и паролю, затем по JWT. Авторизация отдельно проверяет роль и принадлежность объекта. Недостаточно проверить только роль teacher: иначе любой преподаватель получил бы возможность менять результаты чужой олимпиады. Поэтому общая функция check_owner сравнивает teacher_id и id пользователя.

Профиль изменяется отдельной схемой без полей role, is_active и token_version. Лишние поля отклоняются. В публичном протоколе выдаются ФИО, баллы и результат; email, дата рождения и хеш отсутствуют. Проверки выполняются на сервере независимо от HTTP-клиента [10, 19].

На рисунке 1 показаны варианты использования для четырёх актёров. Стрелки зависимостей между прецедентами не введены, поскольку основные операции представлены самостоятельными вариантами использования. Доступ к общим публичным операциям сохраняется и у зарегистрированных пользователей.

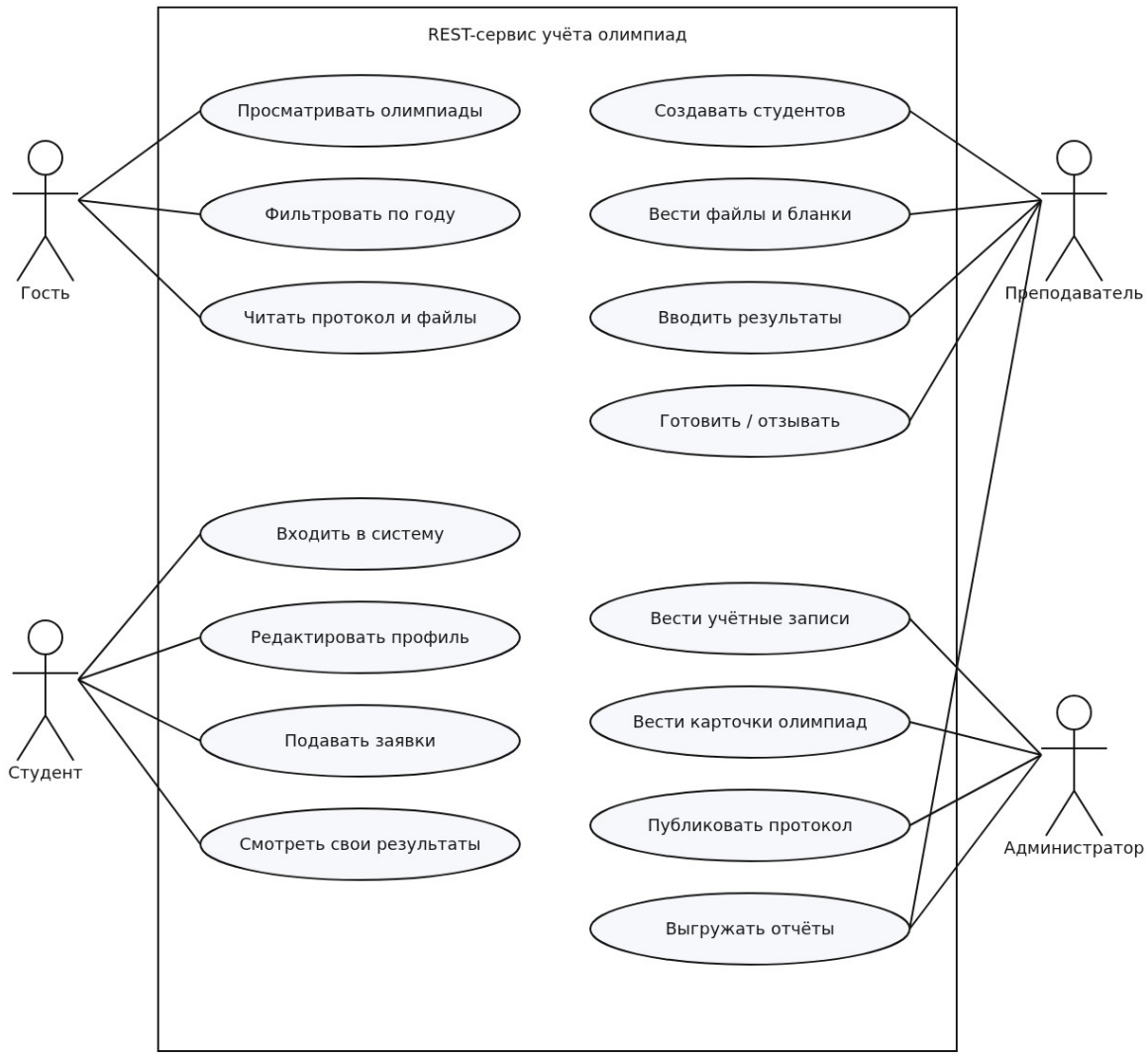


Рисунок 1 — Варианты использования REST-сервиса

2.4 Внутренняя архитектура

Структура показана на рисунке 2. FastAPI получает запрос и проверяет тело через схемы Pydantic из schemas.py. Обработчик в main.py получает сессию SQLAlchemy и текущего пользователя. Пароли и JWT вынесены в auth.py, модели — в models.py, подключение — в database.py, Word/Excel — в reports.py.

Модели SQLAlchemy описывают четыре таблицы. Они не передаются клиенту напрямую без выбора полей: для основных объектов применяются выходные схемы, для протоколов и заявок явно составляются словари. Это позволяет отделить физическую структуру таблицы users от открытого ответа API.

Сессия БД создаётся на запрос и закрывается после его завершения. При нарушении уникальности выполняется rollback и возвращается 409. Изменения статуса, заявки и результата используют блокировку строки олимпиады. Все версии работают с одной базой; номер API не обозначает отдельную копию данных.

Модульный подход остаётся простым: маршруты — обычные функции, классы используются для ORM и схем данных. Повторяющиеся проверки владельца, статуса и сохранения находятся рядом с маршрутами в main.py. Такое разделение соответствует назначению частей программы [8, 11, 16, 21]. Используется одна PostgreSQL, распределённое хранение в данном проекте не требуется [2, 9].

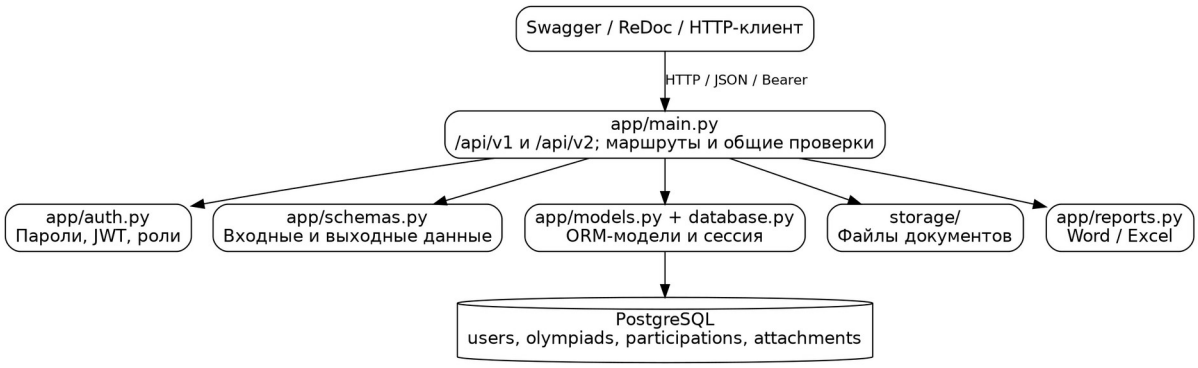


Рисунок 2 — Структура приложения и движение данных

2.5 Проектные решения по тестированию

Проверки должны обнаруживать ошибки в действиях и правах, а не только подтверждать запуск процесса. Для этого выбран pytest, FastAPI TestClient и отдельная PostgreSQL-база. Тесты создают минимальный набор аккаунтов и событий, выполняют HTTP-запросы и проверяют коды, структуру ответа и сохранённое состояние [5, 7, 13, 14, 15, 17].

Положительные случаи дополнены неверными паролями/JWT, отсутствующими записями, повторной заявкой, неверными баллами и датами, чужим преподавателем и публикацией неподготовленного протокола. Для файлов проверены содержимое, расширение, размер, замена и удаление; Word/Excel повторно открываются библиотеками. Подход к ошибочным и граничным значениям соответствует задачам систематического тестирования [5, 7, 13, 14, 15, 17].

Каждый тест использует внешнюю транзакцию и savepoint. Вызов commit внутри маршрута не делает тестовые данные постоянными: после теста внешняя транзакция откатывается. Каталог загрузок заменяется временным. Тестовая БД имеет имя с суффиксом _test, что дополнительно препятствует случайному запуску на рабочей базе.

Критерий завершения — успешный набор тестов, отсутствие известных блокирующих дефектов и покрытие не ниже 80%. Порог задан в pytest.ini и автоматически проверяется при запуске. План и матрица представлены отдельно в TEST_PLAN.md, действия — в TEST_CASES.md, фактический отчёт — в TEST_REPORT.md. Для обязательных дополнений выполнен цикл падающих тестов, реализации и повторного прогона; TDD для всей предыдущей истории не заявляется.

3 ПРОЕКТНАЯ ЧАСТЬ

3.1 Стек технологий

Проверенная среда — Python 3.12.14 и PostgreSQL 17.11. Для обычной установки указаны Python 3.11+ и PostgreSQL 15+. FastAPI 0.115.12 создаёт маршруты и OpenAPI; Pydantic 2.11.7 проверяет данные; SQLAlchemy 2.0.41 описывает ORM; psycopg 3.2.9 подключается к клиент-серверной СУБД [1, 6, 9, 18, 20].

PyJWT 2.10.1 применяется для подписанных токенов HS256. PBKDF2 реализован средствами стандартной библиотеки hashlib: дополнительный пакет для хранения паролей не нужен. python-multipart разбирает формы авторизации и загрузки. python-dotenv читает локальные настройки, не помещая их в исходный код.

Word формируется библиотекой python-docx 1.2.0, Excel — openpyxl 3.1.5. Тесты используют pytest 8.4.1, pytest-cov 6.2.1 и httpx 0.28.1. Рабочие зависимости перечислены в requirements.txt, тестовые — в requirements-dev.txt. Requests и отдельный консольный клиент не требуются.

Используется синхронная SQLAlchemy-сессия и обычные функции def. Для небольшого учебного сервиса это проще для чтения и отладки. PostgreSQL 17 устанавливается как обычная служба операционной системы. Python-приложение работает в venv и подключается к ней через psycopg. Для запуска требуются Python и PostgreSQL.

3.2 Модель и ограничения базы данных

Модель содержит users, olympiads, participations и attachments. На рисунке 3 показаны ключи и связи. Словарь полей приведён в таблицах 2–5. Структура сверена с моделями и настоящей PostgreSQL проверочного запуска. Создание таблиц выполняется Base.metadata.create_all в init_db.py.

Учётная запись хранит общие сведения и роль. Для студента обязательны учебные поля; для преподавателя они могут быть пустыми. Обязательность зависит от роли и проверяется схемой создания. Логин и email уникальны. Поле created_by описывает создателя записи, token_version используется для отзыва токенов.

Олимпиада ссылается на ответственного преподавателя. В заявке есть уникальная пара olympiad_id/student_id. Именно ограничение БД препятствует дублированию при одновременных запросах. Внешние ключи не допускают ссылок на отсутствующие записи. Удаление олимпиады с данными блокируется приложением и связями.

При формировании отчёта баллы хранятся как Numeric(8,2), а не как двоичное число с плавающей точкой. В JSON ответ преобразуется к числу для удобства клиента. Категории результата, роли и переходы статусов проверяются приложением; не все правила продублированы SQL CHECK. Это различие отражено в словаре данных.

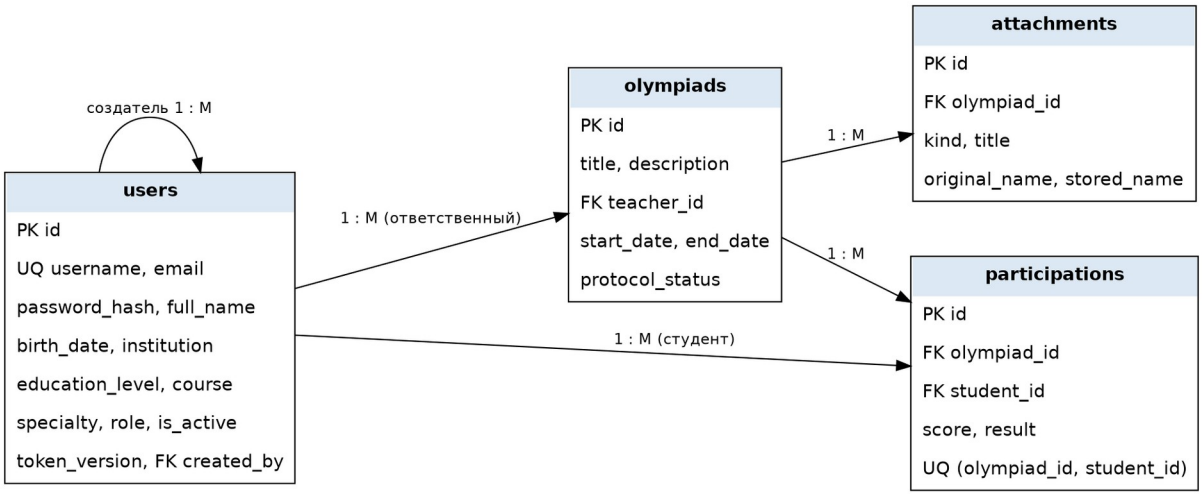


Рисунок 3 — Физическая модель данных PostgreSQL

Состав таблиц и типы столбцов соответствуют app/models.py и PostgreSQL 17. Словарь позволяет сопоставить поля API и физическое хранение.

PK — первичный ключ; FK — внешний ключ; UQ — уникальность. Проверки ролей и большинства входных полей выполняются схемами Pydantic и обработчиками API; ограничения БД перечислены отдельно.

users

Таблица 2 — Словарь данных: users

Ключ	**Поле**	**Тип PostgreSQL**	**NULL**	**Назначение**
PK	id	INTEGER	нет	Идентификатор аккаунта
UQ	username	VARCHAR(50)	нет	Уникальный логин, 3–50 символов
—	password_hash	VARCHAR(256)	нет	PBKDF2-хеш; исходный пароль не хранится
—	full_name	VARCHAR(150)	нет	Фамилия, имя, отчество
—	birth_date	DATE	да	Дата рождения; для студента обязательна
UQ	email	VARCHAR(254)	нет	Уникальная электронная почта
—	institution	VARCHAR(200)	нет	Учебное заведение
—	education_level	VARCHAR(100)	нет	Уровень образования
—	course	INTEGER	да	Курс обучения, 1–6
—	specialty	VARCHAR(150)	нет	Код и название специальности
—	role	VARCHAR(20)	нет	admin / teacher / student
—	is_active	BOOLEAN	нет	Признак действующего аккаунта
—	token_version	INTEGER	нет	Версия токенов для отзыва сеансов
FK → users.id	created_by	INTEGER	да	Кто создал аккаунт; допускается NULL для начального администратора

olympiads

Таблица 3 — Словарь данных: olympiads

Ключ	Поле	Тип PostgreSQL	NULL	Назначение
PK	id	INTEGER	нет	Идентификатор мероприятия
—	title	VARCHAR(200)	нет	Название
—	description	TEXT	нет	Описание
FK → users.id	teacher_id	INTEGER	нет	Ответственный действующий преподаватель
—	start_date	DATE	нет	Дата начала и источник года для фильтра
—	end_date	DATE	нет	Дата окончания, не раньше начала
—	protocol_status	VARCHAR(20)	нет	draft / prepared / published

participations

Таблица 4 — Словарь данных: participations

Ключ	Поле	Тип PostgreSQL	NULL	Назначение
PK	id	INTEGER	нет	Идентификатор заявки
FK → olympiads.id	olympiad_id	INTEGER	нет	Ссылка на мероприятие
FK → users.id	student_id	INTEGER	нет	Ссылка на студента
—	score	NUMERIC(8, 2)	да	Баллы 0…999999.99, два десятичных знака; NULL до ввода
—	result	VARCHAR(20)	да	winner / prize / participant; NULL до ввода

attachments

Таблица 5 — Словарь данных: attachments

Ключ	Поле	Тип PostgreSQL	NULL	Назначение
PK	id	INTEGER	нет	Идентификатор документа
FK → olympiads.id	olympiad_id	INTEGER	нет	Владелец документа — мероприятие
—	kind	VARCHAR(20)	нет	regulation / task / certificate / award
—	title	VARCHAR(150)	нет	Отображаемое название
—	original_name	VARCHAR(200)	нет	Исходное безопасное имя для скачивания
—	stored_name	VARCHAR(100)	нет	Случайное имя физического файла, не выдаётся API

Связи и ограничения

users → olympiads: один преподаватель отвечает за много мероприятий. users → participations: один студент имеет много заявок. olympiads → participations и olympiads → attachments: связи один-ко-многим. users.created_by ссылается на users.id.

В БД действуют уникальности username, email и пары (olympiad_id, student_id), CHECK end_date ≥ start_date, CHECK score ≥ 0, NOT NULL и внешние ключи. Значения ролей, категории результата, верхняя граница баллов, активность ответственного, обязательность учебных полей и переходы статусов проверяются приложением. Ошибки целостности откатываются и возвращают 409.

Удаление олимпиад с зависимостями блокируется. Файловое содержимое хранится вне БД: attachments содержит только метаданные. Схема соответствует третьей нормальной форме в принятом объёме: данные студента не дублируются в каждой заявке; название мероприятия не дублируется в результатах; справочные роли и типы имеют небольшой фиксированный набор и не вынесены в отдельные таблицы.

3.3 Файловая структура и назначение модулей

Корень содержит README.md, requirements.txt, requirements-dev.txt, pytest.ini, database.sql, init_db.py и пример .env. app содержит сервер, tests — автоматические проверки, docs — необходимые документы сдачи, storage — загруженные файлы. Генераторы пояснительной записки и снимков в репозиторий не включены.

В main.py находятся обработчики входа, профиля, пользователей, олимпиад, участия, результатов, протоколов и документов. models.py содержит четыре ORM-класса; schemas.py — входные и выходные модели; auth.py — пароль и JWT; database.py — сессию и настройки; reports.py — табличные выгрузки.

Корневое приложение монтирует api_v1 и api_v2. Каждый получает общие маршруты через include_router, а список олимпиад регистрируется отдельно для нужного контракта. Документация создаётся на лету: /api/v1/openapi.json и /api/v2/openapi.json. Статические копии этих схем в комплект не добавлены.

Тесты сгруппированы по учётным записям, олимпиадам, протоколам, вложениям, контракту и обязательным требованиям. conftest.py создаёт отдельное соединение PostgreSQL, транзакции и фикстуры; зависимости переопределяются у обоих подприложений. Это позволяет запускать тесты без отдельного HTTP-процесса.

3.4 Алгоритм записи на олимпиаду

Схема записи приведена на рисунке 4. Сервер получает Bearer-токен и проверяет существование активного студента. Роль teacher или admin не даёт права записать себя как участника. Мероприятие ищется по идентификатору; отсутствие записи даёт 404.

До добавления заявки блокируется строка олимпиады. Проверяется статус draft и дата окончания. Условие даты позволяет подать заявку в последний день включительно. Затем создаётся Participation с пустыми баллами и результатом. Успех возвращает 201 и идентификатор заявки.

Если такая пара студент–олимпиада уже существует, ограничение UNIQUE вызывает ошибку целостности. Общая функция save откатывает транзакцию и преобразует ошибку в 409. Проверка только предварительным SELECT была бы недостаточна: два одновременных запроса могли бы оба не найти заявку до вставки.

Отмена заявки выполняется отдельным DELETE. Студент может удалить только свою заявку, преподаватель — в своей олимпиаде, администратор — любую при draft. После подготовки протокола состав участников фиксируется. Эти правила проверяются положительными и отрицательными тестами.

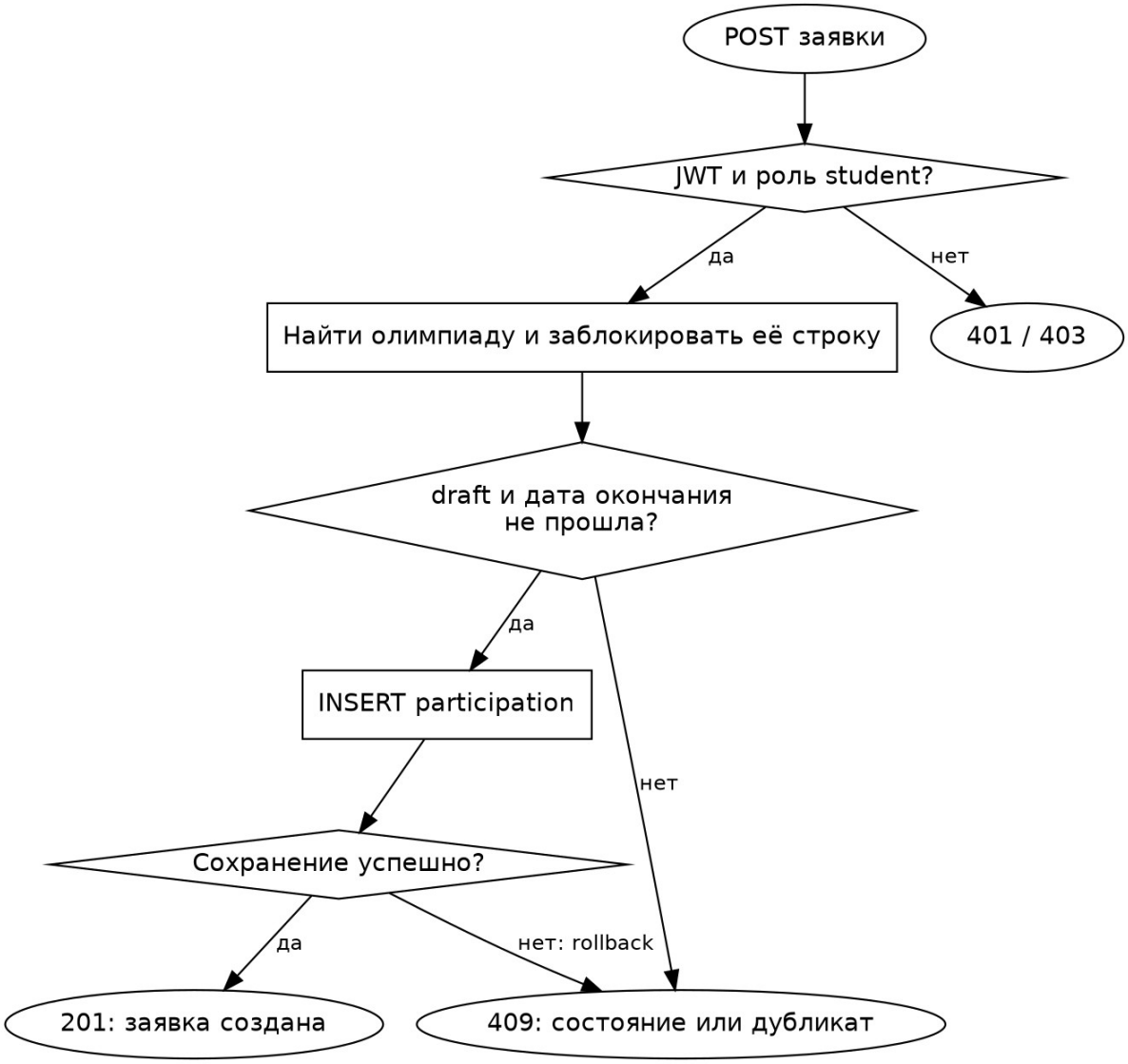


Рисунок 4 — Алгоритм обработки заявки

3.4.1 Подготовка, отзыв и публикация протокола

Модель состояний показана на рисунке 5. Исходное состояние — draft. В нём преподаватель заполняет баллы и категорию результата каждого участника. Список сортируется по убыванию баллов, однако это не назначает призовое место автоматически.

При переходе draft → prepared проверяются ответственность пользователя, наступление даты окончания, наличие участников и заполненность всех результатов. Только после этого сохраняется prepared. Повторный такой же переход и попытка пропустить подготовку приводят к 409.

Ответственный может отозвать prepared в draft. Тогда редактирование становится доступно снова. Переход prepared → published выполняет только администратор. При published публичный маршрут начинает возвращать результаты, а персональный список студента перестаёт скрывать баллы.

Готовность Word/Excel не равнозначна публикации: преподаватель может скачать черновую выгрузку для проверки. Её заголовок содержит текущий статус. Гость получает протокол только через публичный маршрут, который до публикации возвращает 404. Факт появления результата после публикации проверен HTTP-сценарием.

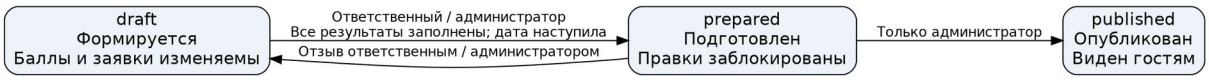


Рисунок 5 — Состояния и переходы протокола

3.4.2 Аутентификация и отзыв токенов

Схема проверки защищённого запроса приведена на рисунке 6. Пароль проверяется через PBKDF2-HMAC-SHA256, соль генерируется отдельно для каждого хеша. Используется 600000 итераций. Результат сравнивается через hmac.compare_digest. Исходный пароль и JWT-ключ в ответы API не включаются.

JWT содержит sub, iat, exp и ver. Принимается только HS256 с настроенным ключом и обязательными полями [6]. Затем из PostgreSQL читается пользователь. Неактивность, отсутствие аккаунта или неверный token_version дают 401.

Сброс пароля или изменение активности администратором увеличивает token_version. Старые токены сразу перестают проходить проверку. Отдельный серверный выход и самостоятельная смена пароля не добавлены: персональное ТЗ API первого этапа их не требует.

Роль и ответственность проверяются после аутентификации. Поэтому корректный JWT чужого преподавателя не открывает доступ к мероприятию. 401 означает необходимость повторного входа, 403 — отсутствие права при известной личности. Срок токена по умолчанию равен 60 минутам.

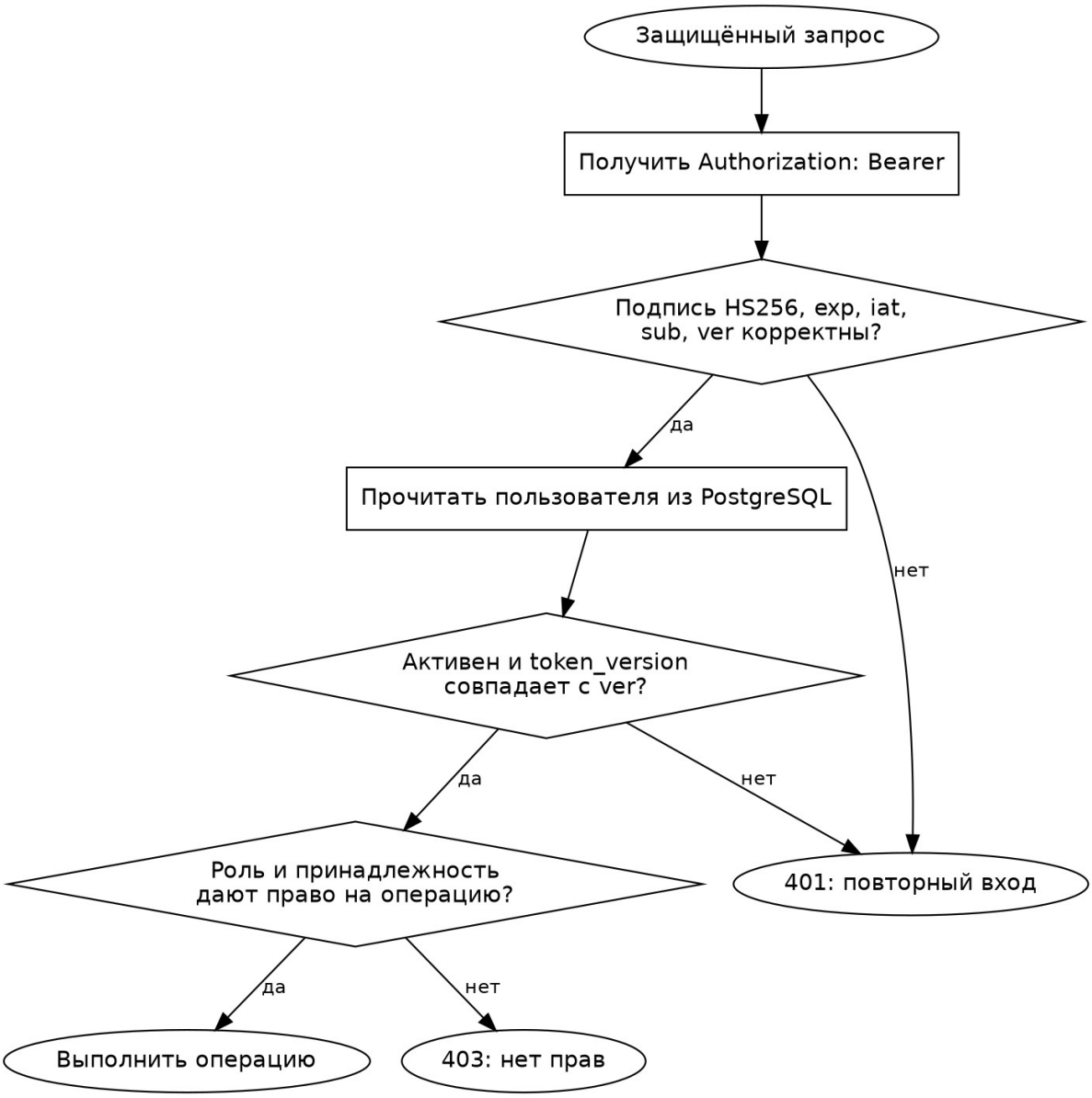


Рисунок 6 — Проверка JWT и прав пользователя

3.5 Документация REST API

Сервис использует JSON и обычные HTTP-методы. Поля запросов и ответов имеют snake_case: teacher_id, start_date, full_name. Токен передаётся в Authorization: Bearer, не в URL. Параметры и схемы доступны в OpenAPI [6].

v1 возвращает массив олимпиад; v2 — объект items, total, page, page_size и поддерживает поиск search. Размер страницы от 1 до 100, номер — от 1. Остальные операции и права общие. v2 расширяет контракт, а не сохраняет ошибки v1.

Swagger: /api/v1/docs и /api/v2/docs. ReDoc: /api/v1/redoc и /api/v2/redoc. Оформление загружается с CDN, поэтому страницам нужен интернет; методы API работают локально. Отдельное руководство docs/API.md содержит операции, роли, поля, коды ошибок и примеры.

В таблице 6 приведён состав операций версии v1. Версия v2 заменяет только GET коллекции олимпиад.

Таблица 6 — Операции REST-сервиса

Метод	Путь	Назначение
GET	/olympiads	Все олимпиады, фильтр по году
POST	/olympiads	Создать олимпиаду
POST	/auth/token	Получить JWT-токен
GET	/users/me	Посмотреть свой профиль
PATCH	/users/me	Изменить свой профиль
GET	/users	Список учетных записей (администратор)
POST	/users	Создать студента или преподавателя
PATCH	/users/{user_id}	Редактировать учетную запись
GET	/olympiads/{olympiad_id}	Карточка олимпиады
PATCH	/olympiads/{olympiad_id}	Изменить поля карточки
PUT	/olympiads/{olympiad_id}	Заменить данные карточки
DELETE	/olympiads/{olympiad_id}	Удалить пустую олимпиаду
POST	/olympiads/{olympiad_id}/ participations	Записаться на олимпиаду
GET	/olympiads/{olympiad_id}/ participations	Заявки для ответственного преподавателя
GET	/users/me/participations	Свои заявки и опубликованные результаты
PUT	/participations/{participation_id}/	Записать баллы и результат

Метод	Путь	Назначение
	result	
DELETE	/participations/{participation_id}	Удалить заявку до подготовки протокола
GET	/protocols	Состояния протоколов (служебный список)
GET	/olympiads/{olympiad_id}/protocol	Опубликованный протокол (общедоступный)
PATCH	/olympiads/{olympiad_id}/protocol	Подготовить, отозвать или опубликовать протокол
GET	/olympiads/{olympiad_id}/ protocol-draft	Просмотреть служебный протокол
GET	/olympiads/{olympiad_id}/ reports/protocol	Скачать протокол Word или Excel
GET	/reports/annual	Годовой отчёт по опубликованным результатам
GET	/olympiads/{olympiad_id}/ attachments	Положение и бланки; задания прошлых лет
POST	/olympiads/{olympiad_id}/ attachments	Прикрепить файл
GET	/attachments/{attachment_id}	Скачать общедоступный документ
PUT	/attachments/{attachment_id}	Заменить файл или название
DELETE	/attachments/{attachment_id}	Удалить документ
GET	/olympiads/{olympiad_id}/ attachments/manage	Все файлы ответственного преподавателя

3.5.1 Примеры запросов и обработка ответа

Примеры JSON и HTTP-запросов приведены в docs/API.md. В Swagger можно получить токен кнопкой Authorize и выполнить запрос с проверкой статуса и тела. При программном обращении следует проверять HTTP-код перед разбором JSON; DELETE 204 имеет пустое тело, выгрузка — двоичное.

В карточке передаются title, description, teacher_id, start_date, end_date. Ответственный должен быть действующим преподавателем. При создании студента передаются все личные и учебные поля ТЗ. Профиль и административное редактирование имеют разные схемы; студент не может повысить себе роль.

Файлы передаются multipart/form-data, обычные данные — JSON. Экспорт возвращает двоичный ответ, который сохраняется через response.content. DELETE при успехе возвращает 204 с пустым телом. Попытка вызвать json() для такого ответа является ошибкой клиента, а не требованием сервиса.

Предметные ошибки имеют поле detail. 401 — вход/JWT, 403 — роль/ответственность, 404 — отсутствие/скрытая запись, 409 — конфликт, 413 — размер файла, 422 — данные или формат. Успех: 200, создание: 201, удаление: 204. Схемы и подробные примеры находятся в OpenAPI и docs/API.md.

3.5.2 Работа пользователей с API

Гость открывает список, задаёт year, читает карточку и опубликованный протокол. Студент входит как student1, изменяет свой профиль, подаёт заявку и получает свои результаты. До публикации баллы в собственном списке скрыты. Все эти действия доступны через Swagger.

Преподаватель входит как teacher1 или teacher2, создаёт студента через POST /users, прикрепляет готовое положение, задания и бланки, вводит score/result, готовит и отзывает prepared-протокол своей олимпиады. Бланк создаётся в текстовом редакторе и загружается; PUT /attachments/{id} позволяет заменить его без изменения номера.

Администратор входит как admin, создаёт преподавателей, правит аккаунты и карточки, получает список /protocols, публикует подготовленные результаты и выгружает годовой отчёт. Демонстрационные пароли: Admin2026!, Teacher2026!, Student2026! соответственно. Студенты student1… student47; все данные учебные.

Рабочий путь: создать/выбрать мероприятие на сегодня → подать заявку студентом → указать баллы преподавателем → prepared → published администратором → получить публичный протокол. Читаемые Word/Excel проверяются отдельными запросами. Снимки API и фактического запроса приведены в приложении 2.

3.5.3 Руководство по развёртыванию

Запуск не использует Docker. Требуются обычная служба PostgreSQL, Python и venv. Команды для Windows/Linux, создание баз и настройка .env приведены отдельно в docs/INSTALL.md. При проверке использованы порты PostgreSQL 5433 и Uvicorn 8001; стандартные значения инструкции — 5432 и 8000.

Администратор PostgreSQL выполняет database.sql: создаются роль olympiad и две базы — рабочая и тестовая. После установки зависимостей из .env.example создаётся .env, генерируется JWT_SECRET длиной от 32 символов. Команда python init_db.py создаёт таблицы и по 50 учебных записей в пустой базе, повтор не меняет непустую.

Основные команды: python -m venv .venv; активировать окружение; python -m pip install -r requirements.txt; python init_db.py; python -m uvicorn app.main:app --reload. После запуска открыть / и /api/v1/docs, выполнить список, вход и профиль. Проверка процесса без запросов недостаточна.

Для тестов устанавливаются requirements-dev.txt и задаётся отдельная TEST_DATABASE_URL с окончанием _test. Команда python -m pytest создаёт тестовые таблицы, выполняет проверки и контролирует покрытие. Неверные адрес, порт или пароль базы исправляются в .env. Снимок запущенного API приведён в приложении 2 и руководстве установки.

3.5.4 Хранение данных и резервное копирование

Информация находится в двух местах: строки — в каталоге данных PostgreSQL, документы — в storage. Для восстановления нужны обе части одного состояния. Копирование только исходников не сохраняет пользовательские загрузки, а копирование только storage не восстанавливает заявки и права. Служба PostgreSQL управляется средствами операционной системы.

Перед согласованным резервным копированием Uvicorn останавливается. База выгружается pg_dump в custom-формате, storage архивируется отдельно. Восстановление выполняется в новую пустую базу, чтобы не затронуть рабочую. После него проверяются вход, список событий и скачивание нескольких файлов.

В учебной установке PostgreSQL принимает локальные подключения с проверкой пароля. Для эксплуатации с реальными персональными данными потребуются отдельные настройки TLS, доступа, резервного хранения и ограничения частоты входа. Нагрузочная пригодность для большого числа пользователей данным набором проверок не подтверждена.

Скрипт create_all создаёт отсутствующие таблицы, но не изменяет уже существующие столбцы. При дальнейшем изменении модели нужен отдельный процесс миграции. Автоматическое удаление базы и перезаполнение не применяется как способ обновления проекта.

3.6 Результаты тестирования

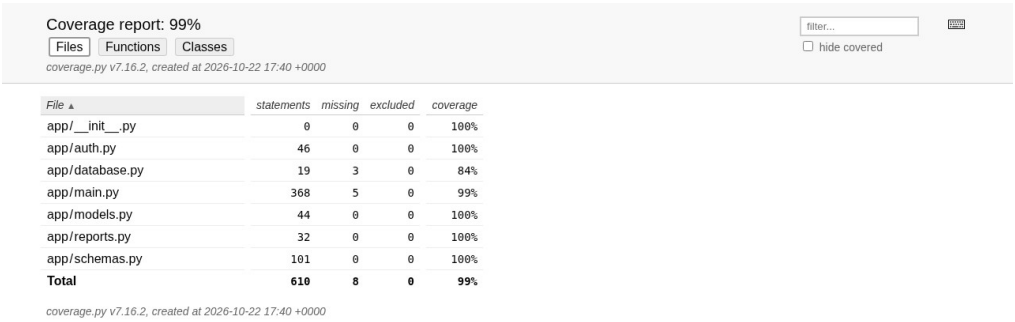
Итоговый автоматический прогон 22.10.2026: 72 теста, успешно 72, ошибок и пропусков 0. Время начала — 17:40:08 UTC (22:40:08 по Екатеринбургу), длительность 16,65 с. Покрытие строк app — 98,69% (602 из 610), main.py с маршрутами и общими проверками — 98,64% (363 из 368). Требуемые 80% достигнуты.

Тесты проверили операции учётных записей, карточки, отбор и версии списка, роли, принадлежность преподавателя, заявки, результаты, переходы статусов, документы и экспорт. В бинарных отчётах проверено фактическое содержимое таблиц, а не только HTTP-код. В Excel отдельно проверено сохранение строки, начинающейся со знака формулы, в безопасном текстовом виде.

Через настоящий Uvicorn выполнены входы трёх ролей, обе версии документации, создание отдельной олимпиады, заявка, 95,5 балла, подготовка, публикация и четыре Word/Excel-выгрузки. До публикации гость получил 404, студент — null; после публикации результат открыт. Выполнен публичный GET из Swagger. Проверочные записи затем удалены из отдельной базы.

В pytest учтён пакет app, поэтому показатель покрытия не относится ко всем файлам репозитория. Наполнение дополнительно проверено отдельным тестом: все четыре таблицы, четыре вида документа и открытие 50 DOCX. Нагрузочные испытания, длительная устойчивость и пользовательские Django-страницы не проверялись.

Сопоставление с обязательными требованиями выявило отсутствие полного PUT карточки, замены бланка по ID, достаточного заполнения и тестов в первоначальной простой ветке. Они дополнены. Реестр наблюдавшихся проблем и закрытие приведены в приложении 1; подробный фактический отчёт — в docs/TEST_REPORT.md.



Coverage report: 99%

Files | Functions | Classes

coverage.py v7.16.2, created at 2026-10-22 17:40 +0000

filter...

hide covered

File ▲	statements	missing	excluded	coverage
app/__init__.py	0	0	0	100%
app/auth.py	46	0	0	100%
app/database.py	19	3	0	84%
app/main.py	368	5	0	99%
app/models.py	44	0	0	100%
app/reports.py	32	0	0	100%
app/schemas.py	101	0	0	100%
Total	**610**	**8**	**0**	**99%**

coverage.py v7.16.2, created at 2026-10-22 17:40 +0000

Рисунок 7 — Отчёт фактического автоматического прогона

4 ГЛОССАРИЙ

В таблице 7 приведены термины и сокращения, используемые в работе.

Таблица 7 — Термины и сокращения

Термин	**Определение**
APM	Автоматизированное рабочее место.
API	Программный интерфейс для взаимодействия приложений.
REST	Подход к организации HTTP-взаимодействия с ресурсами.
HTTP	Протокол запросов и ответов клиента и сервера.
JSON	Текстовый формат обмена структурированными данными.
JWT	Подписанный токен с набором утверждений и сроком действия.
ORM	Отображение объектов программы на таблицы базы данных.
БД / СУБД	База данных / система управления базами данных.
PK / FK	Первичный ключ / внешний ключ.
CRUD	Создание, чтение, обновление и удаление записей.
OpenAPI	Машиночитаемое описание операций HTTP API.
Swagger / ReDoc	Интерфейсы чтения и проверки OpenAPI.
PBKDF2	Функция получения хеша пароля с солью и повторными вычислениями.
Транзакция	Группа операций с подтверждением или откатом.
Пагинация	Получение списка частями заданного размера.
Фикстура	Подготовка и освобождение состояния теста.
Покрытие	Доля строк или ветвей кода, выполненных при проверках.
TDD	Разработка через цикл тест — реализация — рефакторинг.

5 СПИСОК ИСТОЧНИКОВ

1. Beaulieu A. Learning SQL. 3rd ed. O’Reilly Media, 2020.

2. Brewer E. CAP Twelve Years Later: How the “Rules” Have Changed // Computer. 2012. Vol. 45, no. 2. P. 23–29.

3. Chen P. P.-S. The Entity-Relationship Model—Toward a Unified View of Data // ACM Transactions on Database Systems. 1976. Vol. 1, no. 1. P. 9–36.

4. Codd E. F. A Relational Model of Data for Large Shared Data Banks // Communications of the ACM. 1970. Vol. 13, no. 6. P. 377–387.

5. Dijkstra E. W. The Humble Programmer // Communications of the ACM. 1972. Vol. 15, no. 10. P. 859–866.

6. FastAPI. Официальная документация: зависимости, безопасность, подприложения и тестирование. URL: https://fastapi.tiangolo.com/.

7. Fenton N. E., Neil M. Software Metrics: Successes, Failures and New Directions // Journal of Systems and Software. 1999. Vol. 47, no. 2–3. P. 149–157.

8. Fowler M. Refactoring: Improving the Design of Existing Code. 2nd ed. Addison-Wesley, 2018.

9. Kleppmann M. Designing Data-Intensive Applications. O’Reilly Media, 2017.

10. Lampson B. W. Hints for Computer System Design // ACM SIGOPS Operating Systems Review. 1983. Vol. 17, no. 5. P. 33–48.

11. Martin R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.

12. Matthes E. Python Crash Course. 3rd ed. No Starch Press, 2023.

13. McCabe T. J. A Complexity Measure // IEEE Transactions on Software Engineering. 1976. Vol. SE-2, no. 4. P. 308–320.

14. Myers G. J., Sandler C., Badgett T. The Art of Software Testing. 3rd ed. Wiley, 2011.

15. Ntafos S. C. A Comparison of Some Structural Testing Strategies // IEEE Transactions on Software Engineering. 1988. Vol. 14, no. 6. P. 868–874.

16. Parnas D. L. On the Criteria To Be Used in Decomposing Systems into Modules // Communications of the ACM. 1972. Vol. 15, no. 12. P. 1053–1058.

17. Percival H. J. W. Test-Driven Development with Python. 2nd ed. O’Reilly Media, 2017.

18. Ramalho L. Fluent Python. 2nd ed. O’Reilly Media, 2022.

19. Saltzer J. H., Schroeder M. D. The Protection of Information in Computer Systems // Proceedings of the IEEE. 1975. Vol. 63, no. 9. P. 1278–1308.

20. Silberschatz A., Korth H. F., Sudarshan S. Database System Concepts. 7th ed. McGraw-Hill, 2019.

21. Sommerville I. Software Engineering. 10th ed. Pearson, 2016.

22. ГОСТ 2.105–95. Единая система конструкторской документации. Общие требования к текстовым документам. Применение по предоставленной методичке.

23. Методические рекомендации по выполнению курсового проекта ПМ.05 / сост. А. В. Грибова. Екатеринбург: ГАПОУ СО «ЕМК», 2026.

24. Общие правила оформления курсовой работы. Раздел 3. Предоставленный PDF, без выходных данных.

25. Техническое задание «Пеннер (1)». Создание и тестирование АРМ учёта участия студентов в олимпиадном движении. Предоставленный материал, 2026.

ЗАКЛЮЧЕНИЕ

В рамках первого этапа разработан REST-сервис для обработки сведений об участии студентов в олимпиадах и конкурсах профессионального мастерства. Определены четыре роли и модель PostgreSQL из четырёх связанных таблиц. Реализованы создание учётных записей, карточки мероприятий, заявки, результаты, документы и формирование Word/Excel.

Публикация отделена от подготовки протокола. До неё гость и студент не получают незавершённые результаты. Преподаватель ограничен своими мероприятиями, администратор управляет учётными записями и публикует подготовленные итоги. Валидация и ограничения БД препятствуют некорректным датам, повторным заявкам и ряду ошибочных переходов.

Автоматические проверки подтвердили основные операции, ограничения доступа и форматы. После обязательных дополнений достигнут минимум данных по 50 записей и порог покрытия 80%. v2 отличается пагинацией и поиском. Структура остаётся простой: несколько серверных файлов, тесты и только необходимая документация.

Первый этап API реализован и проверен в учебном окружении. Перечень требований и связи с проверками приведены в отдельных документах тестирования. Неизвестные ФИО руководителя, точная группа, номер студенческого, обозначение и подписи оставлены для заполнения по реальным сведениям; вымышленные реквизиты не подставлены.

ПРИЛОЖЕНИЕ 1 — РЕЕСТР ДЕФЕКТОВ И ОГРАНИЧЕНИЙ

Таблица 8 — Выявленные проблемы

ID	Проблема	Важность	Шаги и факт	Исправление / статус
B01	Нет полного PUT карточки	Средняя	PUT от admin в v1/v2: 405 вместо 200	Добавлен; обе версии проверены, закрыт
B02	Нет изменения бланка по ID	Высокая	PUT грамоты от ответственного: 405	Добавлен; ID сохранён, старый файл удалён, закрыт
B03	Недостаточный объём данных	Средняя	3 пользователя, 2 олимпиады, 1 заявка, 0 документов	По 50 записей, 50 DOCX, повтор проверен; закрыт
B04	Тестов нет в проекте	Средняя	В первоначальной ветке отсутствует tests/	Добавлены тесты, зависимости и порог покрытия; закрыт

B01–B04 относятся к первоначальной простой ветке: два обязательных PUT возвращали 405, объём данных был недостаточен, тесты не входили в проект. Для новых функций сначала выполнены падающие проверки, затем реализация и успешная регрессия. v2 добавляет пагинацию и поиск; обязательные исправления действуют для обеих версий.

Критических незакрытых функциональных дефектов в проверенном наборе не выявлено. Реестр ограничен реально наблюдавшимися ответами и сопоставлением файлов с требованиями. Выполненные проверки не являются доказательством многодневной устойчивости сервиса.

ПРИЛОЖЕНИЕ 2 — ПРИМЕРЫ РАБОТЫ API

На рисунках 8–10 показан работающий API ветки simple-student: Swagger, фактический ответ GET и вторая версия. Ключ JWT и токены на снимках отсутствуют. Инструкции, запросы и статусы представлены отдельно в docs/API.md и docs/INSTALL.md; дополнительные демонстрационные файлы в комплект не включены.



Рисунок 8 — Swagger первой версии работающего сервиса

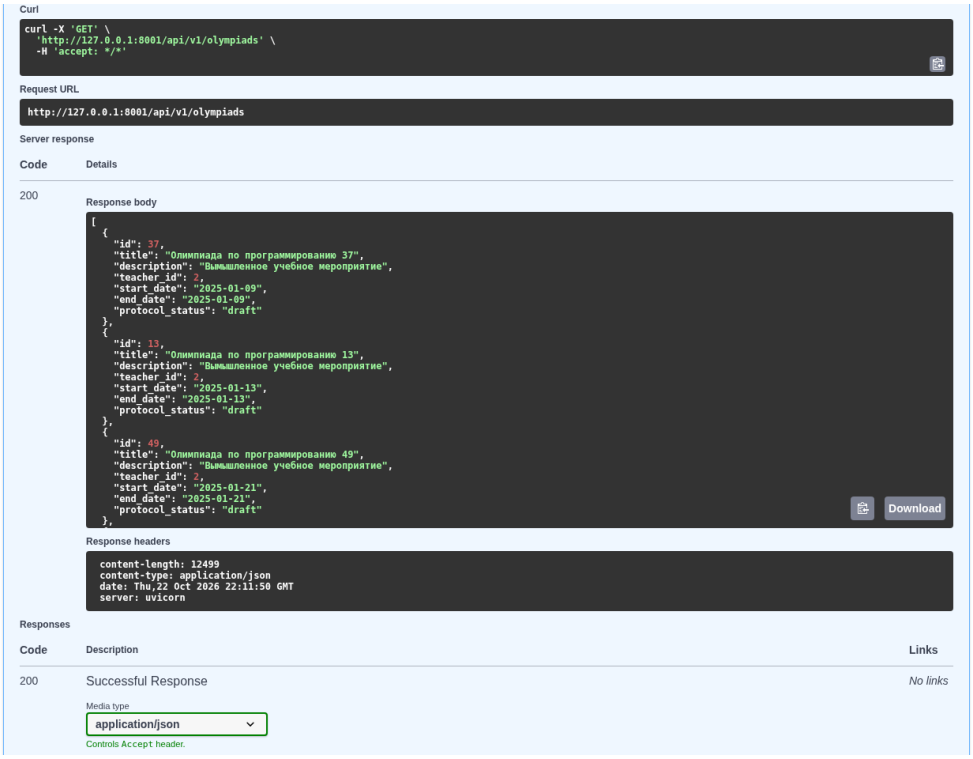


Рисунок 9 — Выполнение GET-запроса из Swagger

Публичные запросы выполняются без авторизации. Для изменяющих операций необходимо войти через Authorize. Код ответа, заголовки и тело позволяют проверить фактический результат, как в презентации requests.



Рисунок 10 — Документация второй версии

У второй версии те же предметные операции и права. Список возвращает items, total, page, page_size; доступны поиск, отбор по году и проверка размера страницы. Контракт описан в docs/API.md.

Фактический HTTP-сценарий выполнен 22.10.2026 в отдельной базе с 50 олимпиадами. Пройдены вход, заявка, баллы, положение, подготовка, публикация и четыре отчёта. В таблице 9 приведена строка результата проверочного студента.

Таблица 9 — Строка проверочного опубликованного протокола

№	ФИО	Баллы	Результат
1	Студент Учебный Номер 1	95.5	Победитель

```
GET /api/v1/olympiads?year=2026
Authorization для публичного списка не требуется.
GET /api/v2/olympiads?page=1&page_size=10
POST /api/v1/auth/token (форма username и password)
GET /api/v1/users/me (Authorization: Bearer <JWT>)
```

Примеры создания студента, карточки, результата, публикации, загрузки и замены документа приведены в отдельном руководстве API.md. Установка и подключение к PostgreSQL — в INSTALL.md; подготовка, сценарии и результаты тестирования разделены на TEST_PLAN.md, TEST_CASES.md, TEST_REPORT.md.